

Task 04: Shopping App

Intern name

Assigned by

Start date

Deadline (date, time, timezone)

01 Objective

Build a Flutter shopping app with a product catalog, persistent cart, simulated checkout and personal order history. Reuse the authentication concepts from Task 03 and focus on state management, accurate totals and reliable order submission.

02 Required screens

- Login / Sign Up: email-password authentication and logout.
- Product Catalog: image, name, price, title search, category filter and price sorting.
- Product Details: image, description, category, price and Add to Cart.
- Cart: product lines, quantity controls, removal and order totals.
- Checkout: delivery information, order summary and Place Order.
- Order Confirmation / History: order ID, timestamp, items, totals and delivery details.

03 Catalog and navigation

- Create a bundled JSON catalog with at least 12 sample products across 3 categories. Each product needs a unique ID, name, description, category, image asset and price in integer paisa.
- Search, category filtering and price sorting must work together. Show a clear no-results state and a fallback for missing images.
- Use Provider for shared app state. Prevent unauthenticated access to checkout and order history. Sign-out must clear visible user data.

Task 04: Shopping App

Cart behavior | Checkout | Technical requirements

04 Cart and calculations

- Adding the same product again increases its quantity instead of creating a duplicate row. Allow quantities from 1 to 10; provide a separate Remove action.
- Persist each signed-in user's cart locally, keyed by UID, and restore it after restart. Switching accounts must not show another user's cart.
- Calculate subtotal from price x quantity. Use a fixed PKR 200 delivery fee for a non-empty cart; delivery is free when subtotal is PKR 5,000 or more. Empty carts have zero totals.
- Store all monetary values as integer paisa and display PKR with two decimal places. Total equals subtotal plus delivery; no tax or discount is required.

05 Checkout and order submission

- Collect full name, mobile number, street address and city. Trim required text fields. Accept a local 11-digit Pakistan mobile number beginning with 03; explain the required format.
- Use a simulated Cash on Delivery option and label checkout as a demo. No real payment, bank connection or shipment should occur.
- Disable checkout for an empty cart. Disable Place Order while saving. Use a stable order ID for each submission attempt and reuse it on retry to avoid duplicate orders.
- Save the order in Firestore before showing confirmation or clearing the cart. On a failed or unconfirmed write, preserve the cart and show a useful retry/pending message.
- Order history must show newest orders first. Save item names, unit prices and quantities as snapshots so old orders do not change with the catalog.

06 Data and access rules

Use `users/{uid}/orders/{orderId}`. Store items, delivery information, subtotalPaisa, deliveryPaisa, totalPaisa, createdAt and status (placed). Use a server timestamp. Firestore rules must allow only the matching authenticated owner to create/read orders and deny client update/delete operations.

Keep models, services, providers and screens separate. Include rules in the repository. Client-calculated prices are acceptable for this training demo only; document that a real store requires trusted server-side price and stock validation.

Task 04: Shopping App

Acceptance checks | Submission | Evaluation

07 Acceptance checks

- Add the same product twice and confirm one cart row with quantity 2. Check quantity limits, removal and empty-cart behavior.
- Use fixture products priced PKR 1,000 and PKR 1,500. Two of the first plus one of the second must give subtotal PKR 3,500, delivery PKR 200 and total PKR 3,700.
- Set the subtotal to exactly PKR 5,000 and verify delivery is free. Confirm all totals update immediately after cart changes.
- Restart the app and verify the cart remains. Switch from account A to B and verify carts and order histories stay separate.
- Place an order, verify confirmation/history and a cleared cart. Tap repeatedly and retry after a network interruption; the same attempt must not create duplicate orders.
- Using the Rules Playground or emulator, verify owner create/read succeeds; another user and an unauthenticated user are denied; client update/delete is denied.
- Test invalid delivery information, no search results, missing images and small-screen keyboard overflow.

08 Submission checklist

Submit source code as a Git repository link or ZIP, an installable APK, and a 3-5 minute demonstration. Include the sample JSON and image assets, Firestore rules, and evidence of the access-rule checks.

README must include setup steps, Flutter version, packages, Firebase configuration steps, data structure, calculation rules, known limitations and any AI assistance. Do not commit passwords or service-account private keys.

09 Evaluation / 100 points

Catalog, search, filters and navigation	15
Cart behavior, persistence and calculations	25
Checkout reliability and order history	25
Access rules and account isolation	15
UI, validation and code quality	10
Documentation and demonstration	10